

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»

Группа Р3211

Дисциплина: Вычислительная математика

ЛАБОРАТОРНАЯ РАБОТА №2

по теме: «Численное решение нелинейных уравнений и систем»

Вариант 11

Выполнил:
Михайлов Петр Сергеевич

Преподаватель:
Малышева Татьяна Алексеевна

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Порядок выполнения работы	3
3	Рабочие формулы используемых методов	3
4	Графики функций на исследуемом интервале	4
4.1	Локализация корней нелинейного уравнения	4
5	Заполненные таблицы вычислительной части 1	5
5.1	Уточнение крайнего правого корня методом половинного деления	5
5.2	Уточнение крайнего левого корня методом секущих	5
5.3	Уточнение центрального корня методом простой итерации	6
6	Решение системы нелинейных уравнений	7
6.1	Графическое отделение корней	7
6.2	Теоретические сведения	7
6.3	Вычислительный процесс	8
6.4	Результаты вычислений	9
7	Листинг программы	10
8	Результаты выполнения программы при различных исходных данных	14
8.1	Успешное нахождение корней нелинейного уравнения	14
8.2	Решение системы нелинейных уравнений	15
8.3	Обработка некорректных входных данных	15
9	Выводы	17

1 Цель работы

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения и системы нелинейных уравнений, выполнить программную реализацию методов.

2 Порядок выполнения работы

1. Отделить корни заданного нелинейного уравнения графически или аналитически.
2. Определить интервалы изоляции корней.
3. Уточнить корни нелинейного уравнения с точностью $\epsilon = 10^{-2}$ выбранными методами.
4. Отделить корни заданной системы нелинейных уравнений графически.
5. Решить систему нелинейных уравнений заданным методом с точностью до 0,01.
6. Выполнить программную реализацию численных методов.
7. Оформить отчет по результатам вычислительной и программной реализации.

3 Рабочие формулы используемых методов

В рамках лабораторной работы применялись следующие численные методы:

- **Метод половинного деления (дихотомии):**

$$x_i = \frac{a_i + b_i}{2} \quad (1)$$

Условие выбора нового интервала: если $f(a_i) \cdot f(x_i) < 0$, то $b_{i+1} = x_i$, иначе $a_{i+1} = x_i$.

- **Метод секущих:**

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k) \quad (2)$$

- **Метод простой итерации (для нелинейного уравнения):** Уравнение приводится к эквивалентному виду $x = \varphi(x)$, где $\varphi(x) = x + \lambda f(x)$. Значение λ выбирается исходя из условия сходимости $\max |\varphi'(x)| < 1$. Итерационный процесс:

$$x_{k+1} = \varphi(x_k) \quad (3)$$

- **Метод Ньютона (для систем нелинейных уравнений):** Для системы $\begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases}$ строится матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix} \quad (4)$$

На каждом шаге решается СЛАУ относительно приращений $\Delta x, \Delta y$:

$$J(x_k, y_k) \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} f(x_k, y_k) \\ g(x_k, y_k) \end{pmatrix} \quad (5)$$

Новые приближения: $x_{k+1} = x_k + \Delta x, y_{k+1} = y_k + \Delta y$.

4 Графики функций на исследуемом интервале

4.1 Локализация корней нелинейного уравнения

Задано нелинейное уравнение (Вариант 11):

$$f(x) = 4,45x^3 + 7,81x^2 - 9,62x - 8,17 = 0 \quad (6)$$

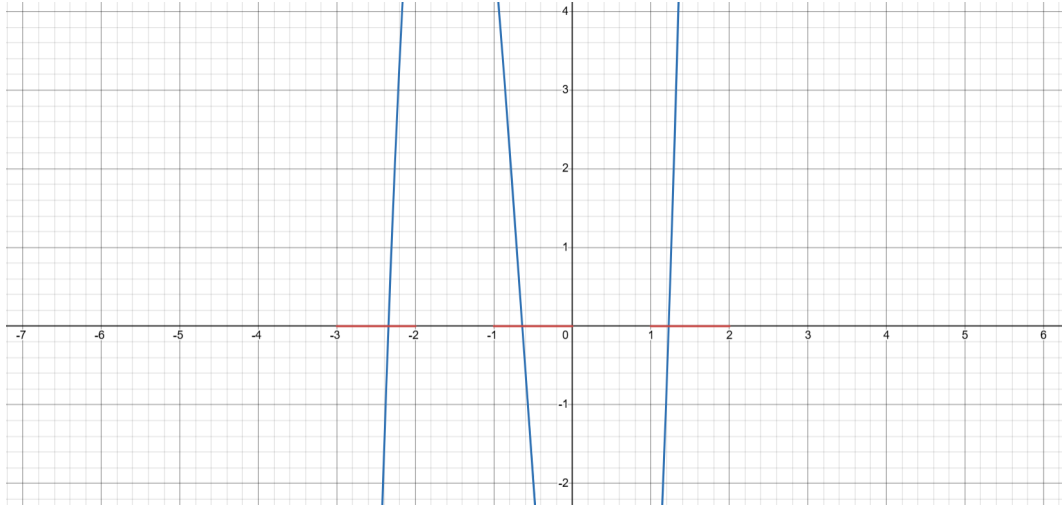


Рис. 1: График функции $f(x) = 4,45x^3 + 7,81x^2 - 9,62x - 8,17$

Найдем экстремумы функции аналитически, приравняв производную к нулю:

$$f'(x) = 13,35x^2 + 15,62x - 9,62 = 0 \quad (7)$$

Корни производной (точки локального экстремума): $x \approx -1,616$ (максимум, $f(x) \approx 8,99$) и $x \approx 0,446$ (минимум, $f(x) \approx -10,51$). Поскольку значения функции в точках экстремумов имеют разные знаки, уравнение имеет ровно 3 действительных корня.

Из графика и аналитического расчета определяем интервалы изоляции:

- Крайний левый корень: $x \in [-3; -2]$
- Центральный корень: $x \in [-1; 0]$
- Крайний правый корень: $x \in [1; 2]$

5 Заполненные таблицы вычислительной части 1

Точность вычислений: $\epsilon = 10^{-2}$.

5.1 Уточнение крайнего правого корня методом половинного деления

Идея метода заключается в том, что начальный интервал изоляции корня делится пополам, в результате чего получается начальное приближение к корню $x_0 = \frac{a_0+b_0}{2}$. Затем вычисляется значение функции $f(x_0)$. В качестве нового интервала выбирается та половина отрезка ($[a_0, x_0]$ или $[b_0, x_0]$), на концах которого функция имеет разные знаки. Другая половина отрезка, на которой функция не меняет знак, отбрасывается. Рабочая формула метода: $x_i = \frac{a_i+b_i}{2}$. В качестве одного из критериев окончания итерационного процесса можно использовать $|a_n - b_n| \leq \epsilon$.

Для интервала $[1; 2]$ функция меняет знак: $f(1) = -5,530$, $f(2) = 39,430$.

№ шага	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ a - b $
1	1,000	2,000	1,500	-5,530	39,430	9,991	1,000
2	1,000	1,500	1,250	-5,530	9,991	0,699	0,500
3	1,000	1,250	1,125	-5,530	0,699	-2,772	0,250
4	1,125	1,250	1,188	-2,772	0,699	-1,129	0,125
5	1,188	1,250	1,219	-1,129	0,699	-0,237	0,063
6	1,219	1,250	1,234	-0,237	0,699	0,226	0,031
7	1,219	1,234	1,227	-0,237	0,226	-0,007	0,015

Таблица 1: Уточнение корня методом половинного деления

Ответ: Крайний правый корень $x \approx 1,227$.

5.2 Уточнение крайнего левого корня методом секущих

Данный метод упрощает метод Ньютона, заменяя производную $f'(x)$ разностным приближением: $f'(x_i) \approx \frac{f(x_i)-f(x_{i-1})}{x_i-x_{i-1}}$. Метод секущих является двухшаговым, то есть новое приближение x_{i+1} определяется двумя предыдущими итерациями x_i и x_{i-1} . Рабочая формула метода: $x_{i+1} = x_i - \frac{x_i-x_{i-1}}{f(x_i)-f(x_{i-1})}f(x_i)$. Критерием окончания процесса выступает $|x_n - x_{n-1}| \leq \epsilon$ или $|f(x_n)| \leq \epsilon$.

Для интервала $[-3; -2]$ заданы начальные приближения $x_0 = -3$ и $x_1 = -2,9$.

№ итерации	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-3,000	-2,900	-2,518	-5,472	0,382
2	-2,900	-2,518	-2,399	-1,593	0,119
3	-2,518	-2,399	-2,351	-0,199	0,048
4	-2,399	-2,351	-2,344	-0,008	0,007

Таблица 2: Уточнение корня методом секущих

Ответ: Крайний левый корень $x \approx -2,344$.

5.3 Уточнение центрального корня методом простой итерации

Суть метода состоит в том, что уравнение $f(x) = 0$ приводится к эквивалентному виду $x = \varphi(x)$ путем выражения x из исходного уравнения. Если непосредственное преобразование не позволяет получить уравнение, удовлетворяющее условиям сходимости, применяется прием введения параметра λ , в результате чего $\varphi(x) = x + \lambda f(x)$. Зная начальное приближение x_0 , очередные приближения вычисляются по рабочей формуле метода: $x_{i+1} = \varphi(x_i)$. Достаточное условие сходимости метода: $\max |\varphi'(x)| \leq q < 1$. В качестве критерия окончания итерационного процесса можно ограничиться условием $|x_n - x_{n-1}| \leq \epsilon$.

На интервале $[-1; 0]$ производная функции отрицательна. Выберем параметр $\lambda = 0,07$ для сведения уравнения к виду $x = \varphi(x) = x + 0,07 \cdot f(x)$. Тогда $\varphi(x) = 0,3115x^3 + 0,5467x^2 + 0,3266x - 0,5719$. На отрезке $\max |\varphi'(x)| \approx 0,3266 < 1$, достаточное условие сходимости выполняется. Выберем начальное приближение $x_0 = -0,5$.

№ итерации	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-0,500	-0,637	-0,019	0,137
2	-0,637	-0,638	-0,010	0,001
3	-0,638	-0,639	0,005	0,001

Таблица 3: Уточнение корня уравнения методом простой итерации

Ответ: Центральный корень $x \approx -0,639$.

6 Решение системы нелинейных уравнений

Задана система нелинейных уравнений:

$$\begin{cases} f_1(x, y) = \operatorname{tg}(xy + 0, 2) - x^2 = 0 \\ f_2(x, y) = x^2 + 2y^2 - 1 = 0 \end{cases} \quad (8)$$

6.1 Графическое отделение корней

Для локализации корней построим графики функций. Второе уравнение — это эллипс с полуосями $a = 1$ и $b \approx 0, 707$. Первое уравнение — периодическая функция, напоминающая тангенсоиду.

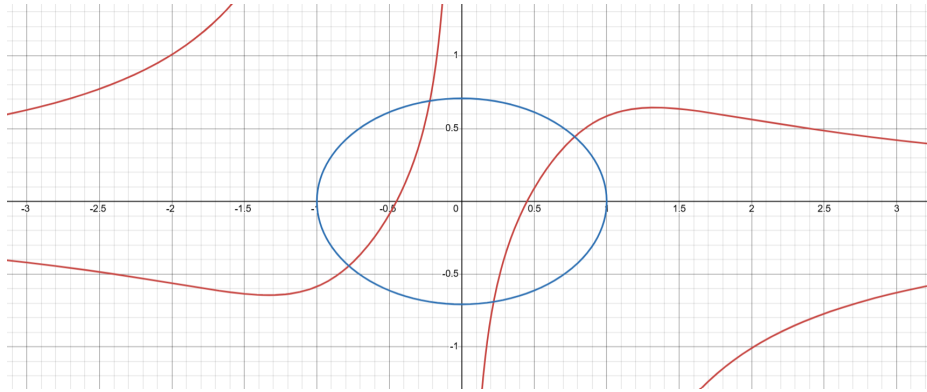


Рис. 2: Графическое отделение корней системы

Примечание: Из графика видно, что система имеет 4 корня (по одному в каждой четверти). В данной работе мы будем уточнять только один корень, находящийся в первой четверти, так как алгоритм нахождения остальных корней идентичен и отличается только выбором начального приближения.

Выберем начальное приближение: $x_0 = 0, 800$, $y_0 = 0, 400$.

6.2 Теоретические сведения

Для уточнения корней используем метод Ньютона. Поправки Δx и Δy на каждой итерации находятся из СЛАУ:

$$\begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -f_1(x_k, y_k) \\ -f_2(x_k, y_k) \end{pmatrix} \quad (9)$$

Частные производные:

- $a_{11} = \frac{\partial f_1}{\partial x} = \frac{y}{\cos^2(xy+0,2)} - 2x$
- $a_{12} = \frac{\partial f_1}{\partial y} = \frac{x}{\cos^2(xy+0,2)}$
- $a_{21} = \frac{\partial f_2}{\partial x} = 2x$
- $a_{22} = \frac{\partial f_2}{\partial y} = 4y$

6.3 Вычислительный процесс

Итерация №1

1. Текущее приближение: $x_0 = 0,800$; $y_0 = 0,400$.

2. Значения функций:

- $f_1(0,8;0,4) = \operatorname{tg}(0,8 \cdot 0,4 + 0,2) - 0,8^2 = \operatorname{tg}(0,52) - 0,64 \approx 0,573 - 0,64 = -0,067$
- $f_2(0,8;0,4) = 0,8^2 + 2 \cdot 0,4^2 - 1 = 0,64 + 0,32 - 1 = -0,040$

3. Значения элементов матрицы Якоби ($\cos^2(0,52) \approx 0,753$):

- $a_{11} = 0,4/0,753 - 1,6 \approx -1,069$
- $a_{12} = 0,8/0,753 \approx 1,062$
- $a_{21} = 2 \cdot 0,8 = 1,600$
- $a_{22} = 4 \cdot 0,4 = 1,600$

4. Решение СЛАУ:

$$\begin{cases} -1,069\Delta x + 1,062\Delta y = 0,067 \\ 1,600\Delta x + 1,600\Delta y = 0,040 \end{cases} \implies \begin{cases} \Delta x_0 \approx -0,019 \\ \Delta y_0 \approx 0,044 \end{cases}$$

5. Новое приближение: $x_1 = 0,781$; $y_1 = 0,444$.

6. Погрешность: $\delta = \max(|\Delta x|, |\Delta y|) = 0,044 > 0,01$. Продолжаем итерации.

Итерация №2

1. Текущее приближение: $x_1 = 0,781$; $y_1 = 0,444$.

2. Значения функций:

- $f_1(0,781;0,444) = \operatorname{tg}(0,547) - 0,610 \approx 0,609 - 0,610 = -0,001$
- $f_2(0,781;0,444) = 0,610 + 0,394 - 1 = 0,004$

3. Значения элементов матрицы Якоби ($\cos^2(0,547) \approx 0,729$):

- $a_{11} = 0,444/0,729 - 1,562 \approx -0,953$
- $a_{12} = 0,781/0,729 \approx 1,071$
- $a_{21} = 2 \cdot 0,781 = 1,562$
- $a_{22} = 4 \cdot 0,444 = 1,776$

4. Решение СЛАУ:

$$\begin{cases} -0,953\Delta x + 1,071\Delta y = 0,001 \\ 1,562\Delta x + 1,776\Delta y = -0,004 \end{cases} \implies \begin{cases} \Delta x_1 \approx -0,002 \\ \Delta y_1 \approx -0,001 \end{cases}$$

5. Новое приближение: $x_2 = 0,779$; $y_2 = 0,443$.

6. Погрешность: $\delta = \max(0,002; 0,001) = 0,002 < 0,01$. Условие точности выполнено.

6.4 Результаты вычислений

Сводная таблица уточнения корней:

k	x_k	y_k	$f_1(x_k, y_k)$	$f_2(x_k, y_k)$	Δx_k	Δy_k	δ
0	0,800	0,400	-0,067	-0,040	-0,019	0,044	0,044
1	0,781	0,444	-0,001	0,004	-0,002	-0,001	0,002
2	0,779	0,443	0,000	0,000	-	-	-

Таблица 4: Результаты итераций метода Ньютона

Ответ: Решение системы при $\epsilon = 0,01$: $x \approx 0,779$, $y \approx 0,443$.

7 Листинг программы

Программный комплекс реализован с использованием современной разделяемой архитектуры: серверная часть (Backend) написана на языке Python с использованием фреймворка FastAPI, а клиентская часть (Frontend) — на JavaScript с использованием библиотеки React.

Полный исходный код проекта доступен в открытом репозитории на GitHub по ссылке: <https://github.com/Axe-On-You/Comp-math/tree/main/labwork2>

Ниже представлены фрагменты кода с реализацией основных численных методов.

```
1 import numpy as np
2 from app.services.functions import EQUATIONS
3
4 def get_interval_warning(f, df, a, b):
5     warnings = []
6
7     if f(a) * f(b) > 0:
8         warnings.append(f"На [{a}, {b}] знаки f(a) и f(b) одинаковы. Не выполняется необходим
9             ое условие существования корня. Корня может не быть или их несколько.")
10
11     points = np.linspace(a, b, 50)
12     df_values = [df(x) for x in points]
13     initial_sign = np.sign(df_values[0])
14     for val in df_values:
15         if np.sign(val) != initial_sign and abs(val) > 1e-12:
16             warnings.append(f"На отрезке [{a}, {b}] функция не монотонна. Не выполняется дост
17                 аточное условие единственности корня.")
18             break
19
20     return warnings
21
22 def check_root_bounds(f, x, a, b, warnings, method_name):
23     """Проверяет положение корня и добавляет развернутое пояснение, если он вне отрезка."""
24     margin = 1e-9
25     if x is not None and not (min(a, b) - margin <= x <= max(a, b) + margin):
26         if f(a) * f(b) < 0:
27             warnings.append(
28                 f"Внимание: корень ({x:.5f}) вне отрезка. {method_name} не обладает локальной
29                     сходимостью "
30                 f"на данном отрезке или выбрано неудачное начальное приближение, из-за чего п
31                     роцесс сошелся к внешнему корню."
32             )
33         else:
34             warnings.append(
35                 f"Внимание: корень ({x:.5f}) вне отрезка. На данном отрезке отсутствуют корни
36                     (или их четное количество), "
37                 f"метод закономерно сошелся к ближайшему внешнему корню."
38             )
39     return warnings
40
41 def solve_by_chord(eq_id, a, b, eps):
42     f = EQUATIONS[eq_id]["f"]
43     df = EQUATIONS[eq_id]["df"]
44     d2f = EQUATIONS[eq_id]["d2f"]
45
46     warnings = get_interval_warning(f, df, a, b)
47
48     if f(a) * d2f(a) > 0:
49         fixed, x_prev = a, b
50     else:
51         fixed, x_prev = b, a
52
53     steps = []
54     res_x = None
55     success = False
56
57     for k in range(1, 101):
58         f_x_prev = f(x_prev)
59         f_fixed = f(fixed)
60         denominator = f_x_prev - f_fixed
61
62         if abs(denominator) < 1e-15:
63             return False, None, k, steps, " ".join(warnings + ["Критическая ошибка: деление н
64                 а 0."])
```

```

59     x_curr = x_prev - f_x_prev * (x_prev - fixed) / denominator
60     delta = abs(x_curr - x_prev)
61     steps.append({"k": k, "x": x_curr, "f_x": f(x_curr), "delta": delta})
62
63
64     if delta < eps and abs(f(x_curr)) < eps:
65         success = True
66         res_x = x_curr
67         break
68     x_prev = x_curr
69
70     warnings = check_root_bounds(f, res_x, a, b, warnings, "Метод хорд")
71     return success, res_x, len(steps), steps, " ".join(warnings) if warnings else None
72
73 def solve_by_newton(eq_id, a, b, eps, initial_x=None):
74     f = EQUATIONS[eq_id]["f"]
75     df = EQUATIONS[eq_id]["df"]
76     d2f = EQUATIONS[eq_id]["d2f"]
77
78     warnings = get_interval_warning(f, df, a, b)
79
80     if initial_x is not None:
81         x_prev = initial_x
82     else:
83         x_prev = a if f(a) * d2f(a) > 0 else b
84
85     steps = []
86     res_x = None
87     success = False
88
89     for k in range(1, 101):
90         df_val = df(x_prev)
91         if abs(df_val) < 1e-15:
92             return False, None, k, steps, " ".join(warnings + ["Ошибка: производная 0."])
93
94         x_curr = x_prev - f(x_prev) / df_val
95         delta = abs(x_curr - x_prev)
96         steps.append({"k": k, "x": x_curr, "f_x": f(x_curr), "delta": delta})
97
98         if delta < eps:
99             success = True
100             res_x = x_curr
101             break
102         x_prev = x_curr
103
104     warnings = check_root_bounds(f, res_x, a, b, warnings, "Метод Ньютона")
105     return success, res_x, len(steps), steps, " ".join(warnings) if warnings else None
106
107 def solve_by_iteration(eq_id, a, b, eps):
108     f = EQUATIONS[eq_id]["f"]
109     df = EQUATIONS[eq_id]["df"]
110
111     warnings = get_interval_warning(f, df, a, b)
112
113     points = np.linspace(a, b, 100)
114     df_values = [df(x) for x in points]
115     max_df = max(abs(v) for v in df_values)
116
117     mid_df = df((a + b) / 2)
118     lambda_val = -1 / max_df if mid_df > 0 else 1 / max_df
119
120     q = max(abs(1 + lambda_val * v) for v in df_values)
121     if q >= 1:
122         warnings.append(f"Условие сходимости МПИ не выполнено (q = {q:.3f} >= 1).")
123
124     x_prev = a
125     steps = []
126     res_x = None
127     success = False
128
129     for k in range(1, 101):
130         try:
131             x_curr = x_prev + lambda_val * f(x_prev)
132
133             if np.isnan(x_curr) or np.isinf(x_curr) or abs(x_curr) > 1e15:
134                 return False, None, k, steps, "Метод разошелся: значения стали слишком ве

```

```

135         лики (Overflow)."
```

```

136     delta = abs(x_curr - x_prev)
137     steps.append({"k": k, "x": x_curr, "f_x": f(x_curr), "delta": delta})
138
139     if delta < eps:
140         success = True
141         res_x = x_curr
142         break
143     x_prev = x_curr
144
145     except OverflowError:
146         return False, None, k, steps, "Ошибка переполнения при вычислении. МПИ не сходитс
147             я на этом интервале."
148
149     warnings = check_root_bounds(f, res_x, a, b, warnings, "Метод простой итерации")
150     return success, res_x, len(steps), steps, " ".join(warnings) if warnings else None
151
152 import numpy as np
153 from app.services.functions import SYSTEMS
154
155 def solve_system_iteration(sys_id, x0, y0, eps):
156     s = SYSTEMS[sys_id]
157     warnings = []
158
159     try:
160         dx1 = s["dphi1_dx"](x0, y0, x0)
161         dy1 = s["dphi1_dy"](x0, y0, x0)
162         dx2 = s["dphi2_dx"](x0, y0, y0)
163         dy2 = s["dphi2_dy"](x0, y0, y0)
164
165         norm1 = max(abs(dx1) + abs(dy1), abs(dx2) + abs(dy2))
166         norm2 = max(abs(dx1) + abs(dx2), abs(dy1) + abs(dy2))
167
168         q = min(norm1, norm2)
169
170         if q >= 1:
171             warnings.append(f"Условие сходимости не выполнено (q = {q:.4f} >= 1). Метод может
172                 расходиться.")
173         else:
174             warnings.append(f"Условие сходимости выполнено: q = {q:.4f}, norm1 = {norm1:.4f},
175                 norm2 = {norm2:.4f}")
176     except Exception:
177         warnings.append("Не удалось рассчитать условие сходимости в начальной точке (возможно
178             , деление на ноль).")
179
180     x_prev, y_prev = x0, y0
181     steps = []
182     success = False
183     res_x, res_y = None, None
184
185     for k in range(1, 101):
186         try:
187             x_curr = s["phi1"](x_prev, y_prev, x0)
188             y_curr = s["phi2"](x_prev, y_prev, y0)
189
190             if np.isnan(x_curr) or np.isnan(y_curr) or abs(x_curr) > 1e10 or abs(y_curr) > 1
191                 e10:
192                 return False, None, None, k, steps, " ".join(warnings + ["Метод разошелся: зн
193                     ачения улетели в бесконечность (Overflow)."])
194
195             dx, dy = abs(x_curr - x_prev), abs(y_curr - y_prev)
196
197             steps.append({
198                 "k": k, "x": x_curr, "y": y_curr,
199                 "delta_x": dx, "delta_y": dy,
200                 "res1": s["f1"](x_curr, y_curr),
201                 "res2": s["f2"](x_curr, y_curr)
202             })
203
204             if max(dx, dy) < eps:
205                 success = True
206                 res_x, res_y = x_curr, y_curr
207                 break

```

```

204         x_prev, y_prev = x_curr, y_curr
205
206     except Exception:
207         return False, None, None, k, steps, " ".join(warnings + ["Критическая ошибка вычи-
                слений (возможно, выход за ОДЗ)."])
208
209     err_msg = " ".join(warnings) if warnings else None
210     if not success:
211         err_msg = (err_msg or "") + " Метод не сошелся за 100 итераций."
212
213     return success, res_x, res_y, len(steps), steps, err_msg.strip() if err_msg else None

```

Листинг 1: Реализация численных методов (фрагмент)

8 Результаты выполнения программы при различных исходных данных

В данном разделе представлены результаты тестирования разработанного веб-приложения на различных наборах входных данных, включая визуализацию процесса и обработку исключительных ситуаций.

8.1 Успешное нахождение корней нелинейного уравнения

На рисунке 3 продемонстрирована работа программы при корректно заданных границах интервала. Программа выводит количество итераций, итоговый результат и график функции с отмеченными границами.

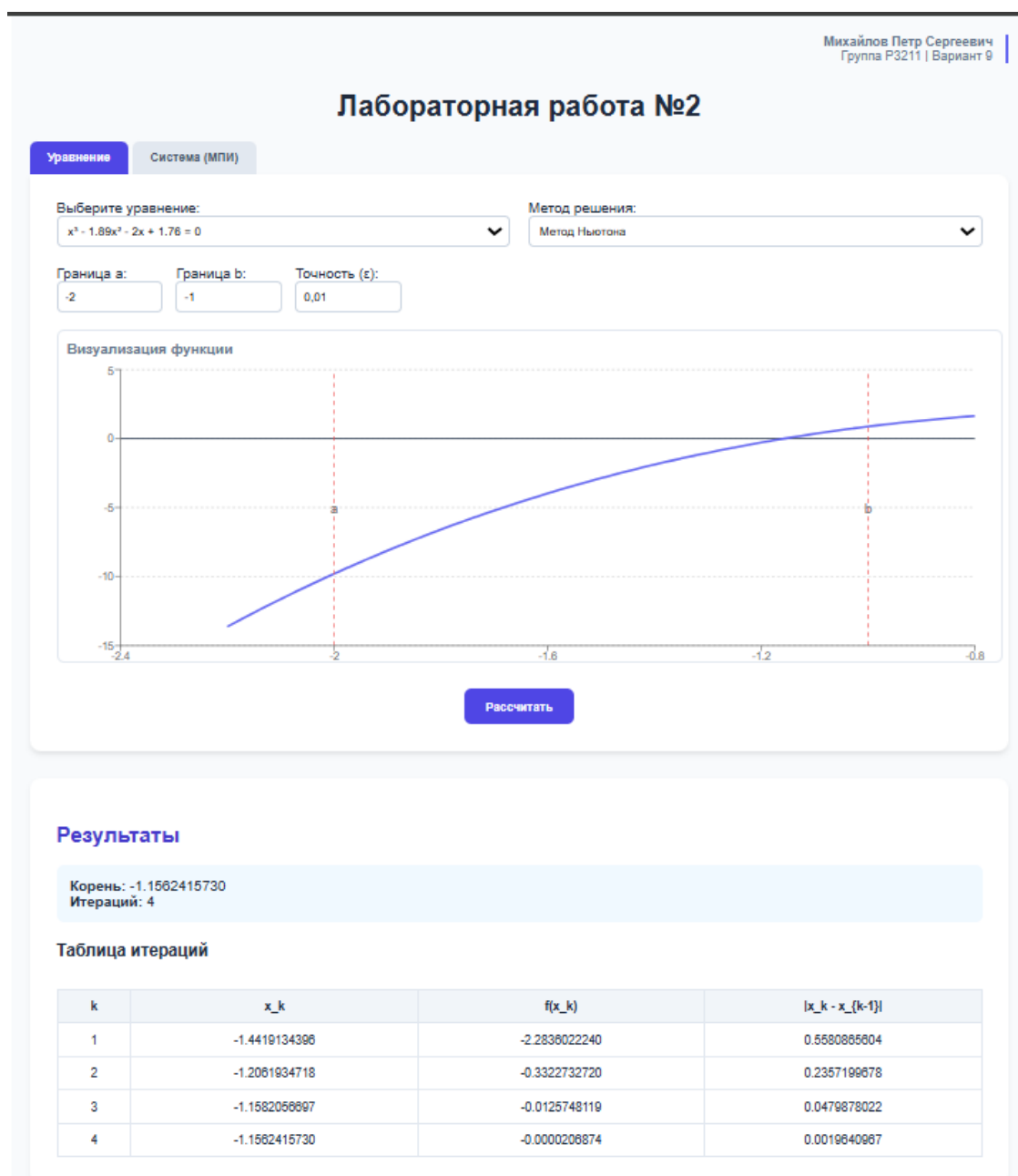


Рис. 3: Результат работы программы: нелинейное уравнение успешно решено

8.2 Решение системы нелинейных уравнений

На рисунке 4 представлен результат решения системы уравнений с визуализацией пересечения графиков (линий уровня), что позволяет наглядно оценить корректность выбранного начального приближения.

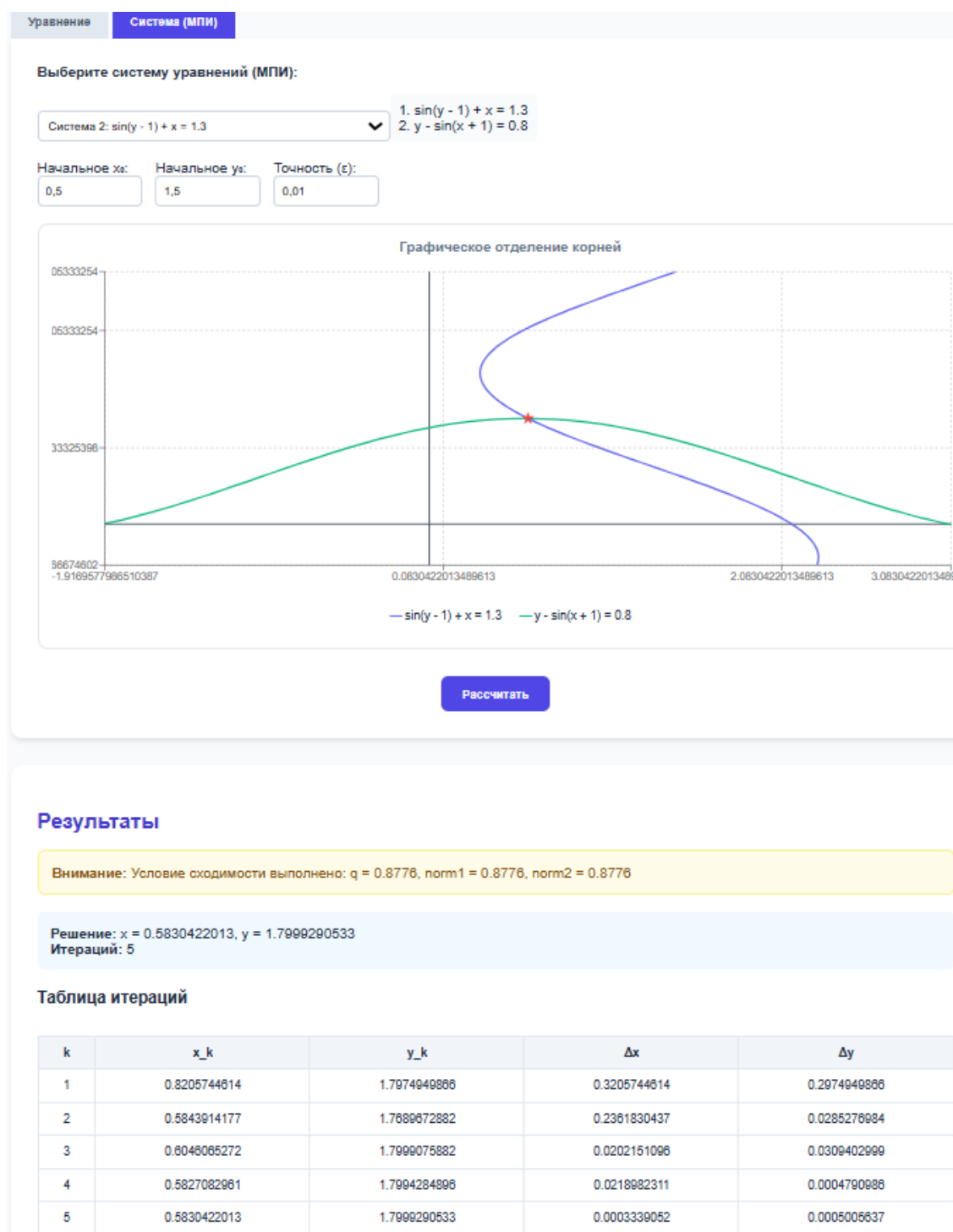


Рис. 4: Результат работы программы: система уравнений успешно решена

8.3 Обработка некорректных входных данных

Программа выполняет валидацию введенных данных и информирует пользователя, если на заданном интервале нет корней (знаки функции на концах отрезка совпадают) или если

метод расходится.

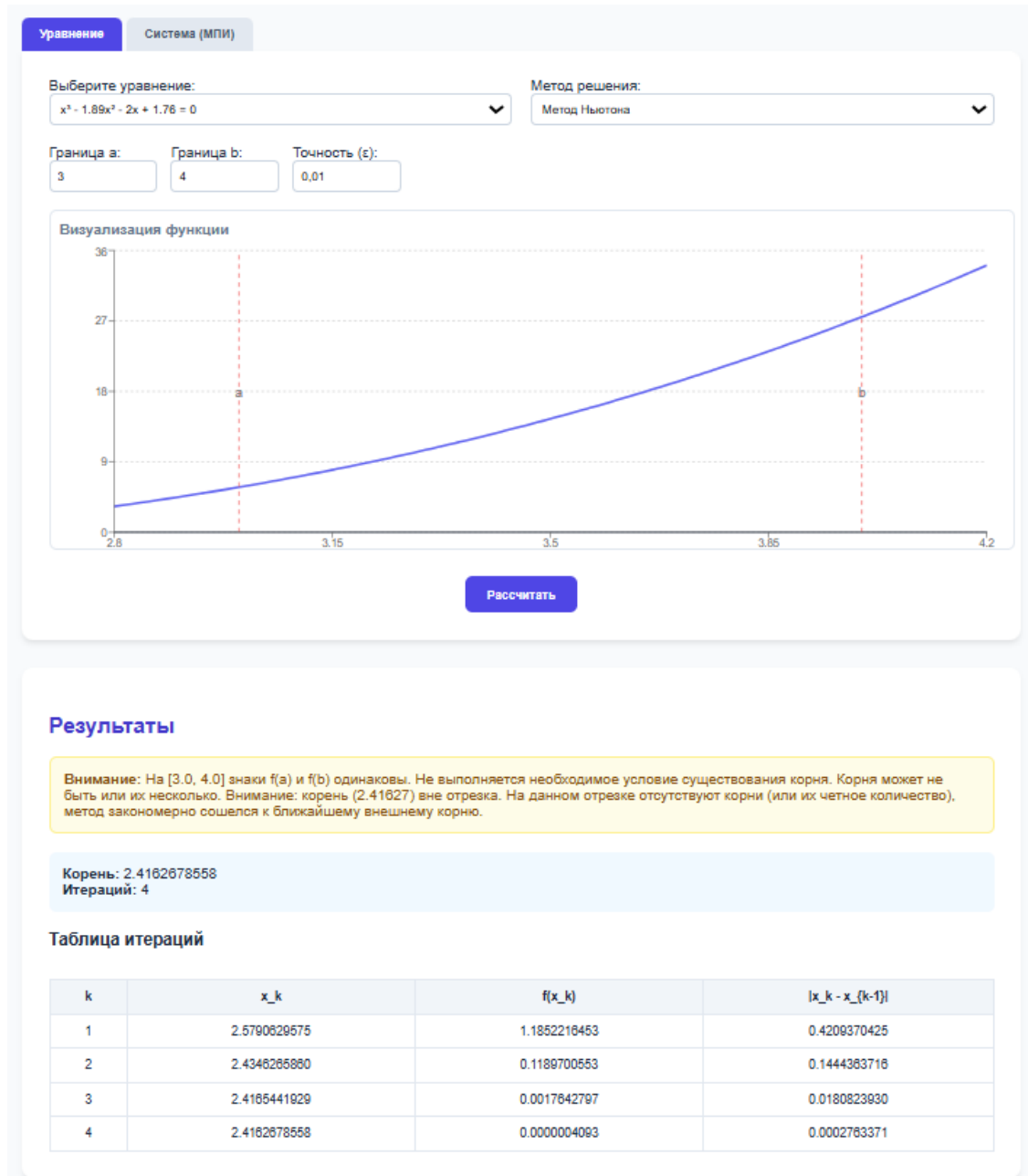


Рис. 5: Реакция программы на отсутствие корня на выбранном интервале

9 Выводы

В ходе выполнения лабораторной работы были изучены и практически применены численные методы решения нелинейных уравнений и систем нелинейных уравнений. Выполнена программная реализация методов половинного деления, секущих, простой итерации, хорд и метода Ньютона.

На основе полученных аналитических и программных результатов можно сделать следующие выводы:

- **Необходимость локализации корней:** Ключевым и обязательным этапом перед применением любого численного метода является локализация (отделение) корней. Использование графического метода позволяет визуально определить количество корней и сузить интервалы поиска, что критически важно для обеспечения сходимости итерационных процессов.
- **Сходимость и скорость методов:** Различные численные методы демонстрируют разную скорость сходимости и имеют разные требования к исходным данным. Метод половинного деления обладает гарантированной сходимостью для непрерывных функций, меняющих знак на интервале, но сходится относительно медленно. Метод Ньютона обладает высокой (квадратичной) скоростью сходимости, однако требует вычисления производной и весьма чувствителен к выбору начального приближения.
- **Особенности метода простой итерации:** Данный метод удобен для программной реализации, так как требует лишь вычисления значения самой функции на каждом шаге. Однако для его успешного применения необходимо предварительно провести математический анализ функции и подобрать параметр λ так, чтобы выполнялось достаточное условие сходимости ($\max |\varphi'(x)| < 1$ на интервале).
- **Решение систем нелинейных уравнений:** Переход от одиночных уравнений к системам существенно усложняет задачу. Численное решение систем (в частности, методом Ньютона) требует на каждом итерационном шаге вычисления элементов матрицы Якоби и решения соответствующей системы линейных алгебраических уравнений (СЛАУ).

Разработанное программное обеспечение успешно решает поставленные задачи, обеспечивает наглядную графическую визуализацию процессов и контролирует корректность входных данных. Совпадение результатов ручных (табличных) расчетов с результатами работы программы подтверждает правильность реализованных численных алгоритмов.